



ACADÉMIE
DE NANTES

Liberté
Égalité
Fraternité



Lycée
**TOUCHARD
WASHINGTON**

Dossier Personnel

BTS CIEL SESSION 2025 – IR E6 – PROJET TECHNIQUE

Gestionnaire de charge de véhicules électrique



Lycée : <u>Touchard-Washington</u>		Ville : LE MANS
Nom du projet :	Gestionnaire de charge de Véhicules électriques	
N° du projet :	TW2	

Projet nouveau	Oui ☐ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	<u>Philippe CRUCHET</u> , Didier Bernard, François Martin, Anthony LE <u>CREN</u> , Jilali <u>KHAMACH</u> , François <u>QUEREC</u>		

Sommaire

Cas D'utilisation.....	3
Piloter la charge.....	3
Surveiller la température du boîtier.....	4
Diagrammes de séquences.....	5
Piloter la charge.....	5
Surveiller la température du boîtier.....	6
Composants matériels.....	6
Horloge temps réel – DS3231.....	6
Relais statique – SSR 20DA.....	7
Capteur de température – DS18S20.....	7
Diagramme de classe.....	8
Protocoles.....	9
WiFi.....	9
WebSocket.....	9
JSON.....	10
Outils.....	12
Visual Studio Code.....	12
LittleFS.....	13
SQLite.....	14
Structures de données.....	14
GitHub.....	15
Fiche de test.....	15
Diagramme de Gantt.....	21
Compétences acquises.....	22
Perspectives d'améliorations.....	23

Cas D'utilisation

Piloter la charge

Description du cas D'utilisation	Le gestionnaire de charge interroge chaque minute la base de donnée afin de détecter les sessions de charge planifiés et de commander automatiquement la charge du véhicule électrique. Il permet à l'utilisateur de forcer manuellement l'état de la charge depuis l'application mobile.
Pré conditions	➤ Horaires de charge. ➤ Horloge temps réel synchronisé via NTP.
Post conditions	➤ Les alertes éventuelles sont enregistrées en base de données.
Scénario principale	Interrogation du calendrier : • À chaque déclenchement de l'alarme minute de l'horloge en temps réel, le système récupère le jour de la semaine (format 1=lundi à 7=dimanche), l'heure et la minute courantes. • La base de données est interrogée afin de détecter une session de charge planifié correspondant à l'heure actuelle. Démarrage de la charge : • Si un début de créneau est détecté, le gestionnaire commande la fermeture du relais. Arrêt de la charge : • Si une fin de créneau est détectée, le gestionnaire commande l'ouverture du relais. Pilotage depuis l'application mobile : • Si une requête d'activation de la marche forcée est reçu, le relais est fermé indépendamment du calendrier. • Si une requête de désactivation de la marche forcée est reçu et qu'aucune session de charge n'est active, le relais est ouvert.
Alternatives	• Dépassement du seuil de température durant la charge : le relais est ouvert immédiatement, la charge est interrompue et une alerte est enregistrée en base de données. Ce cas est traité en détail dans le cas d'utilisation "Surveiller la température du câble et du boîtier".

Surveiller la température du boîtier

Description	Le gestionnaire de charge surveille en permanence la température du boîtier afin de garantir la sécurité de l'installation. En cas de dépassement du seuil thermique configuré, le système coupe immédiatement la charge et enregistre une alerte pour informer l'utilisateur.
Pré conditions	➤ Une température seuil est définie (80°C). ➤ Une charge est en cours (session de charge ou marche forcée).
Post conditions	➤ Le relais est ouvert, la charge est arrêtée. ➤ Une alerte de type surchauffe est enregistrée en base de données. ➤ L'application mobile est informée de l'alerte lors de sa prochaine connexion.
Scénario principale	Lecture de la température : • À chaque tour de boucle, le gestionnaire de charge vérifie si une charge est en cours. • Si c'est le cas, le capteur de température est interrogé. • Le capteur de température effectue la mesure et la valeur relevée est automatiquement comparée au seuil configuré. Dépassement du seuil : • Si un dépassement du seuil est détecté, le gestionnaire commande l'ouverture du relais. • Une alerte de type surchauffe est insérée dans la base de données. Notification de l'alerte : • Lors de la prochaine connexion de l'application mobile, celle-ci envoie une requête. • Le gestionnaire lui retourne l'alerte. • La table correspondante dans la base de données est vidée après l'envoi.
Alternatives	• Capteur non détecté : Si aucun capteur n'est détecté, la surveillance est désactivée et la charge continue sans protection thermique.

Diagrammes de séquences

Suite à la présentation de mes cas d'utilisation et des composants matériels utilisés il est désormais possible de modéliser des diagrammes de séquence complets afin d'illustrer le fonctionnement réel de l'application.

Piloter la charge

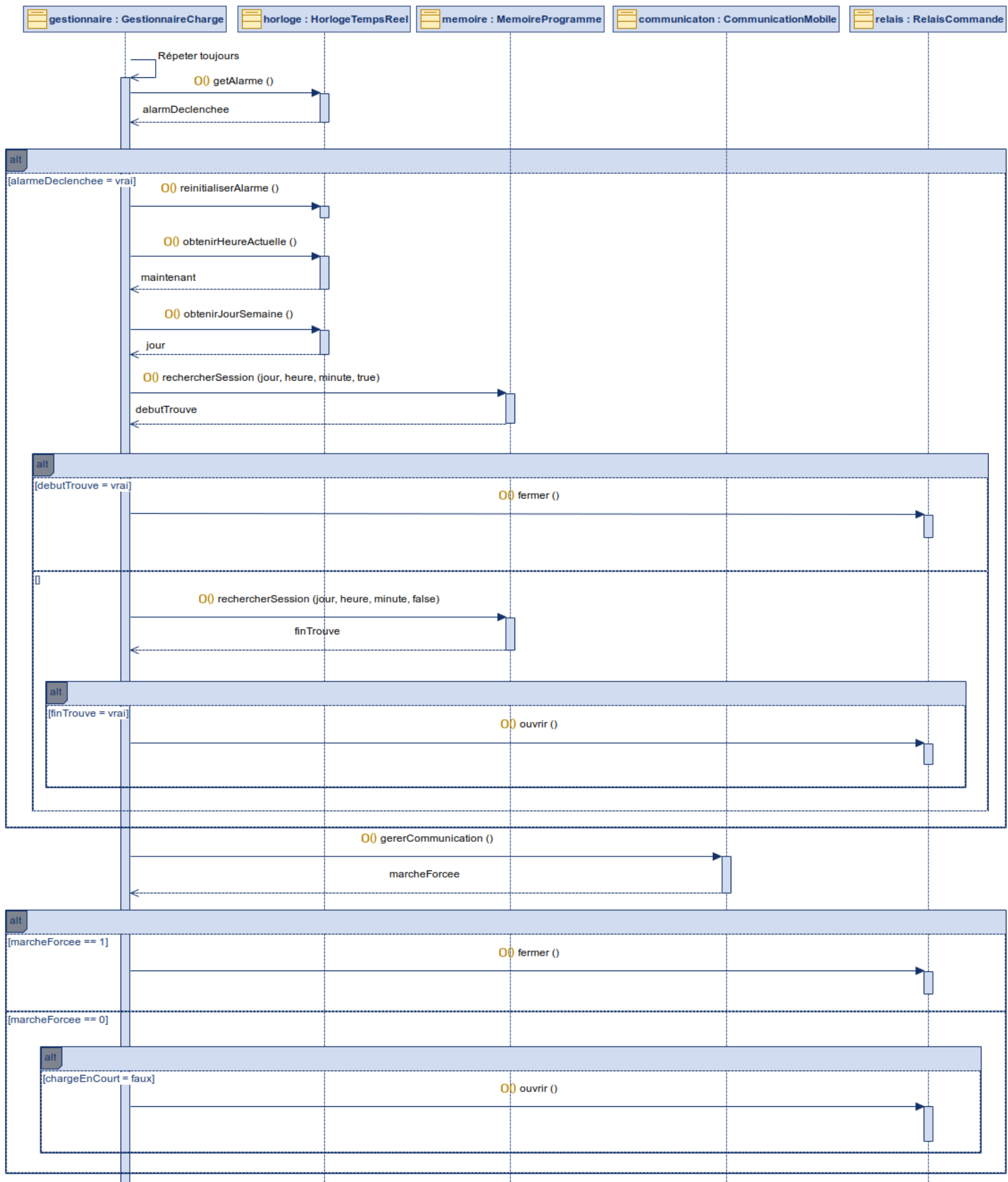


Figure 1: Diagramme de séquence “Piloter la charge”

Surveiller la température du boîtier

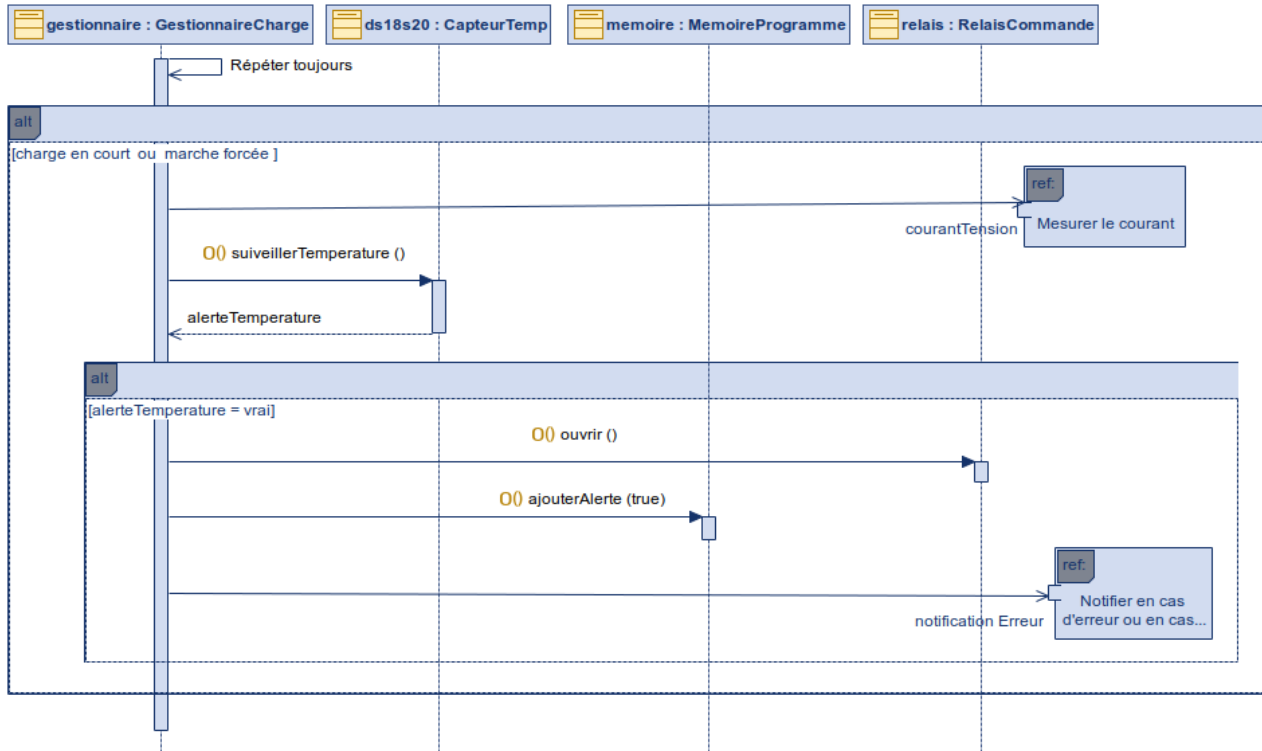


Figure 2: Diagramme de séquence “Surveiller la température du boîtier”

Composants matériels

Horloge temps réel – DS3231

Le DS3231 est un module d'horloge temps réel (RTC) de haute précision communiquant avec l'ESP32 via le protocole I2C, géré grâce à la bibliothèque « RTCLib ». Il maintient une heure exacte de manière autonome grâce à sa pile de sauvegarde intégrée, même en cas de coupure de courant. La caractéristique la plus intéressante pour ce projet est la présence d'alarmes intégrées permettant de générer un signal électrique sur la broche GPIO 23 de l'ESP32 à interval régulier, déclenchant ainsi une interruption matérielle toutes les minutes sans mobiliser le processeur en permanence.

Relais statique – SSR 20DA

Le relais statique est un composant électronique de type tout ou rien (TOR) permettant de commander l'alimentation du véhicule électrique. Contrairement à un relais électromécanique classique, il ne comporte aucune pièce mobile et assure la commutation du circuit de charge par voie purement électronique, ce qui lui confère une durée de vie plus longue, une meilleure fiabilité et un temps de réaction plus court. Il est piloté directement depuis l'ESP32 via la broche GPIO26, et son état est soit ouvert (charge arrêtée) soit fermé (charge en cours). Ce composant est au cœur du système de pilotage de la charge, car c'est lui qui assure physiquement la coupure ou l'établissement du courant vers le véhicule.



Capteur de température – DS18S20

Le DS18B20 est un capteur de température numérique communicant via le protocole OneWire, ce qui lui permet de transmettre ses mesures sur un seul fil de données. Il est placé dans le boîtier de charge afin de surveiller en permanence la température durant les sessions de charge. Une caractéristique particulièrement intéressante pour ce projet est la possibilité de configurer directement sur le composant des seuils d'alarme haute et basse, permettant de déléguer la surveillance thermique au capteur lui-même plutôt que de la gérer logiciellement. Il est piloté directement depuis l'ESP32 via la broche GPIO18. En cas de dépassement du seuil thermique configuré, il permet au gestionnaire de charge de réagir immédiatement en coupant l'alimentation du véhicule afin de garantir la sécurité de l'installation.



Diagramme de classe

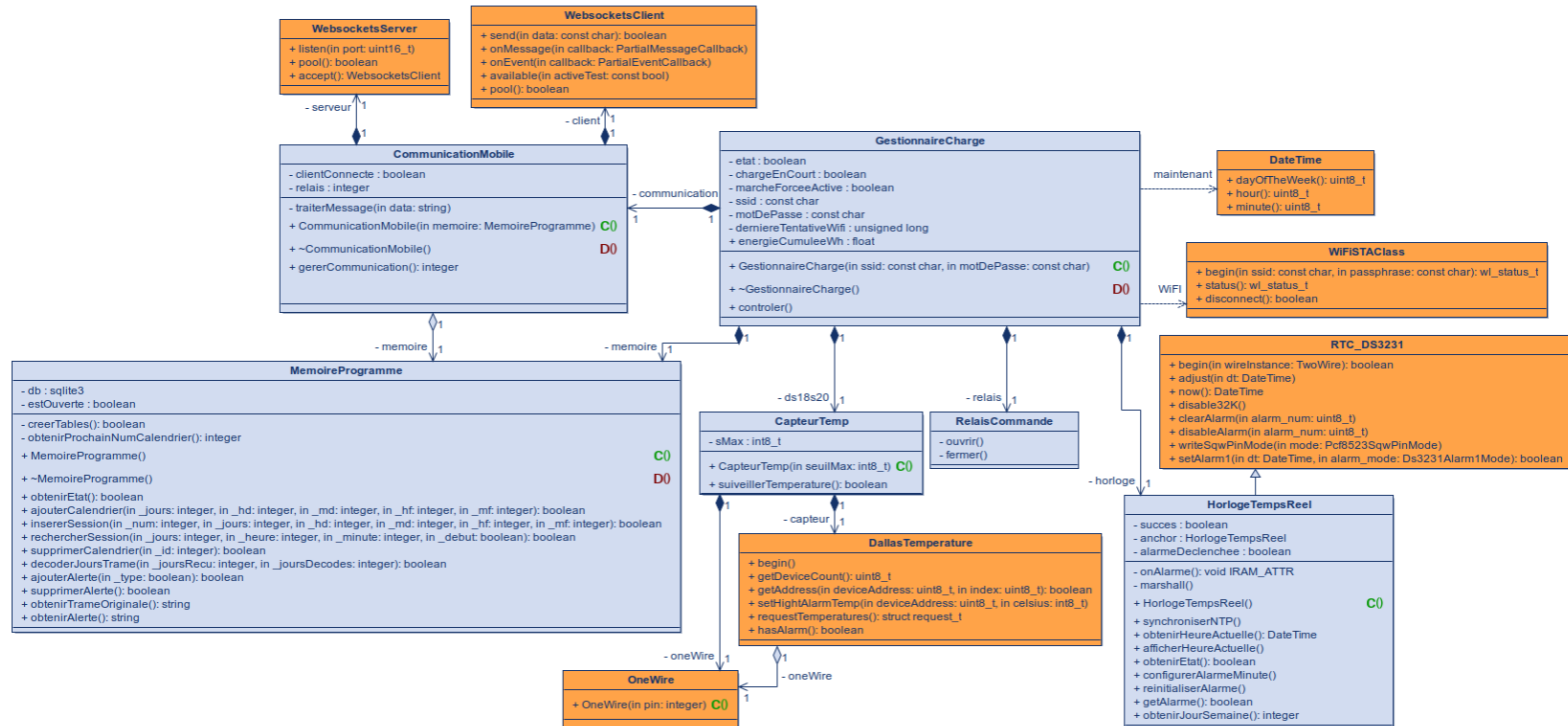


Figure 3: Diagramme de classe

Le choix de nos outils étant finalisé, nous pouvons à présent affiner nos diagrammes afin de les adapter à notre environnement de développement. Ce diagramme de classes intègre ainsi les classes spécifiques gérées par PlatformIO sous Visual Studio Code :

- **WebsocketsServer** : permet de configurer et de gérer un serveur WebSocket sur l'ESP32, d'écouter sur un port spécifique et d'accepter les demandes de connexion du client mobile.
- **WebsocketsClient** : permet de gérer la communication bidirectionnelle avec un client WebSocket connecté, d'envoyer des données et de définir des fonctions de rappel (*callbacks*) pour traiter les messages et événements reçus.
- **WIFISTAClass** : permet de gérer la connectivité Wi-Fi de la carte en mode Station (STA), d'assurer la connexion au point d'accès (SSID et mot de passe) et de surveiller l'état de la liaison réseau.
- **RTC_DS3231** : permet de piloter le module d'horloge temps réel DS3231 via le bus I2C, afin de maintenir l'heure du système à jour, de la synchroniser et de configurer des alarmes matérielles.

- **DateTime** : permet de manipuler, stocker et structurer facilement les données temporelles (heures, minutes, jours de la semaine) récupérées depuis le module RTC.
- **DallasTemperature** : permet de superviser les capteurs de température (type DS18B20), de lancer des requêtes de lecture de température et de configurer des seuils d'alerte thermiques.
- **OneWire** : permet de gérer le protocole de communication matériel "One-Wire" (bus à un seul fil), indispensable pour dialoguer avec le capteur de température Dallas.

Protocoles

WiFi

La communication entre l'ESP32 et l'application mobile repose sur un réseau WiFi local. L'ESP32 intègre nativement un module WiFi, géré via la bibliothèque Arduino qui met à disposition une classe dédiée : « WiFiSTAClass ».

Le WiFi est également utilisé lors du démarrage pour synchroniser l'horloge temps réel DS3231 via le protocole NTP (Network Time Protocol), permettant d'obtenir l'heure exacte depuis un serveur en ligne sans intervention manuelle.

WebSocket

Les WebSockets permettent d'établir une connexion persistante et bidirectionnelle entre le gestionnaire de charge (jouant le rôle de serveur) et l'application mobile (client). Contrairement aux requêtes HTTP classiques, cette technologie évite le besoin d'interroger régulièrement le serveur en maintenant la connexion ouverte. Ainsi, dès que l'application envoie une commande (activation de la marche forcée, modification du calendrier), l'ESP32 traite la requête et retourne immédiatement la réponse sur la même connexion.

Cette solution présente plusieurs avantages :

- Réactivité des commandes : lorsque l'utilisateur active la marche forcée ou modifie un calendrier de charge depuis l'application, l'ESP32 reçoit, exécute la commande et retourne le calendrier mis à jour instantanément.
- Récupération des alertes : lors de la connexion de l'application, celle-ci envoie une requête « obtenirCalendrier » qui déclenche également l'envoi des alertes enregistrées en base de données (surchauffe, surintensité).
- Optimisation des ressources : la connexion WebSocket sur le port 5555 est maintenue ouverte tant que le client est connecté, réduisant ainsi la consommation de bande passante par rapport à des requêtes HTTP répétées.

JSON

En ce qui concerne l'envoi des données, nous avons choisi le format JSON car il est lisible, léger et est donc le plus approprié à notre projet.

Toutes les trames échangées entre l'ESP32 et l'application mobile contiennent obligatoirement un champ « action » qui identifie le type de message. Le reste des champs dépend de l'action concernée.

La structure générale est la suivante : {"action":"<type>", [<champs>]}

Trames envoyées par l'application mobile vers l'ESP32 :

Champs	Nom	Exemple de valeur
Type de message	action	« ajouterCalendrier »
Masque de bits des jours	jours	5
Heure de début	hd	8
Minute de début	md	30
Heure de fin	hf	17
Minute de fin	mf	0

Exemple : {"action":"ajouterCalendrier","jours":5,"hd":8,"md":30,"hf":17,"mf":0}

Champs	Nom	Exemple de valeur
Type de message	action	"supprimerCalendrier"
Identifiant du calendrier	id	3

Exemple : {"action":"supprimerCalendrier","id":3}

Champs	Nom	Exemple de valeur
Type de message	action	"marcheForcee"
Activation du relais	activer	true

Exemple : {"action":"marcheForcee","activer":true}

Champs	Nom	Exemple de valeur
Type de message	action	"obtenirCalendrier"

Exemple : {"action":"obtenirCalendrier"}

Trames envoyées par l'ESP32 vers l'application mobile :

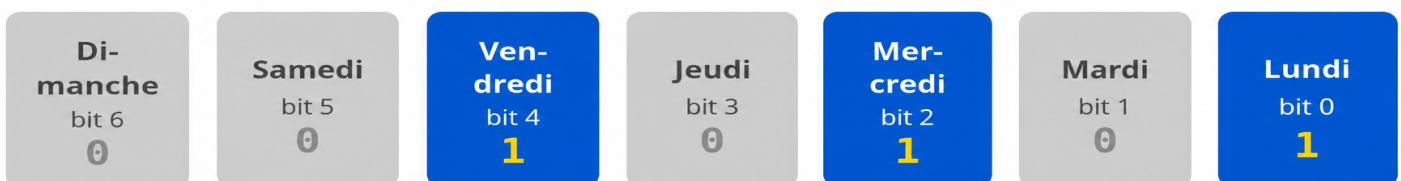
Champs	Nom	Exemple de valeur
Type de message	action	«calendrier»
Masque de bits des jours	jours	5
Heure de début	hd	8
Minute de début	md	30
Heure de fin	hf	17
Minute de fin	mf	0

Exemple : {"action":"ajouterCalendrier","jours":5,"hd":8,"md":30,"hf":17,"mf":0}

Champs	Nom	Exemple de valeur
Type de message	action	"alerte"
Type d'alerte	type	"surchauffe"

Exemple : {"action":"alerte","type":"surchauffe"}

Le champ « jours » présent dans les trames « ajouterCalendrier » et « calendrier » ne représente pas directement un jour de la semaine, mais un masque de bits. Chaque bit de cet entier correspond à un jour de la semaine, du bit 0 pour le lundi au bit 6 pour le dimanche. Si le bit est à 1, le jour est sélectionné ; s'il est à 0, il ne l'est pas. Cette représentation permet d'encoder plusieurs jours simultanément dans un seul entier, ce qui réduit la taille de la trame échangée entre l'application mobile et l'ESP32. Par exemple, pour une charge programmée le lundi, le mercredi et le vendredi, la valeur envoyée sera 21, comme illustré ci-dessous.



Représentation binaire (7 bits) :



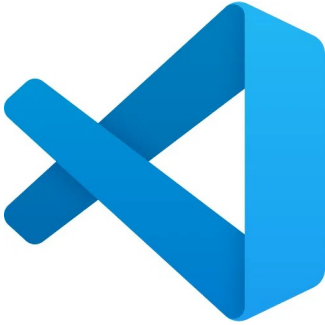
Calcul de la valeur :

$$2^0 + 2^2 + 2^4 = 1 + 4 + 16 = 21$$

{"action": "ajouterCalendrier", "jours": 21, ...}

Outils

Visual Studio Code



Dans ce projet, Visual Studio Code a été utilisé conjointement avec l'extension PlatformIO comme environnement de développement pour l'ESP32. PlatformIO a grandement simplifié le processus de développement en centralisant la compilation, le flashage et le débogage du code C++ directement depuis l'éditeur. Il a également pris en charge la gestion automatique des bibliothèques utilisées dans le projet, évitant ainsi toute configuration manuelle fastidieuse.

Moniteur série

Visual Studio Code, via l'extension PlatformIO, intègre un moniteur série permettant de gérer les sorties console de l'ESP32 en utilisant les fonctions « `Serial.print()` » et « `Serial.printf()` ». Grâce à cette fonctionnalité, nous pouvons afficher les données reçues et envoyées, les erreurs et les informations liées au traitement des données. Une constante « `DEBUGTTEST` » a été mis en place afin de pouvoir activer ou désactiver l'ensemble de ces affichages sans modifier le code fonctionnel.

WebSockets

Les WebSockets sont utilisés pour permettre la transmission des données en temps réel vers l'application Android destinée aux utilisateurs grâce à la classe « `ArduinoWebsockets` ».

Lecture/Écriture JSON

Dans ce projet de gestionnaire de charge, la classe « `ArduinoJson` » est utilisée pour gérer les trames de données échangées entre l'ESP32 et l'application mobile, notamment :

- le décodage et l'interprétation des commandes reçues depuis l'application (ajout de session, suppression, marche forcée),
- et la construction des trames retournées en réponse (calendrier, alertes de surchauffe).

Le format JSON (JavaScript Object Notation) est particulièrement adapté pour représenter les données de charge sous forme structurée, facilement interprétable par l'application mobile Qt/QML.

`ArduinoJson` joue un rôle central dans la gestion des échanges de données structurées entre l'ESP32 et les clients, assurant une communication claire, efficace et fiable dans le système global.

Lecture/Écriture dans une base de données

PlatformIO, via son gestionnaire de bibliothèques intégré, permet d'intégrer facilement la bibliothèque SQLite dans le projet ESP32. Ce stockage permet de conserver de manière persistante les créneaux de charge programmés par l'utilisateur ainsi que les alertes générées par le système, directement dans la mémoire flash de l'ESP32.

Rôle de PlatformIO et Visual Studio Code dans la gestion de la base de données :

- Gestionnaire de dépendances : PlatformIO gère automatiquement l'installation et la mise à jour de la bibliothèque SQLite ainsi que de LittleFS, évitant toute configuration manuelle complexe.
- Téléversement du système de fichiers : PlatformIO intègre un outil dédié permettant de téléverser le système de fichiers LittleFS directement sur l'ESP32, dans lequel est stockée la base de données SQLite.

LittleFS

LittleFS est un système de fichiers léger conçu spécifiquement pour les microcontrôleurs disposant d'une mémoire flash, comme l'ESP32. Contrairement à son prédécesseur SPIFFS, LittleFS offre une meilleure résistance à la corruption des données en cas de coupure de courant inattendue, grâce à un mécanisme de journalisation. Il est particulièrement adapté aux projets embarqués nécessitant un stockage fiable et persistant de fichiers sur la mémoire flash.

Dans ce projet, LittleFS est utilisé comme système de fichiers hôte pour SQLite, permettant de stocker le fichier de base de données directement dans la mémoire flash de l'ESP32, notamment :

- Monter le système de fichiers au démarrage de l'ESP32 afin de rendre la base de données accessible.
- Héberger le fichier de base de données SQLite (/littlefs/programme.db) contenant les tables « SESSIONS » et « ALERTES ».
- Garantir l'intégrité des données stockées même en cas de redémarrage inopiné ou de coupure de courant.
- Permettre à PlatformIO de téléverser le système de fichiers directement sur l'ESP32 via l'outil dédié.

SQLite



SQLite est un système de gestion de base de données relationnelle embarqué, open source, compatible avec le langage SQL. Contrairement aux SGBD classiques comme MySQL ou MariaDB, SQLite ne nécessite pas de serveur dédié : la base de données est stockée directement dans un fichier sur le système de fichiers. Il est

particulièrement adapté aux systèmes embarqués à ressources limitées, comme l'ESP32, grâce à sa légèreté et sa simplicité de mise en œuvre.

Dans ce projet, SQLite est utilisé pour stocker les données du gestionnaire de charge directement dans la mémoire flash de l'ESP32 via LittleFS, notamment :

- Stocker durablement les créneaux de charge définis par l'utilisateur (jours, heures de début et de fin).
- Gérer les alertes de surchauffe ou de surintensité afin qu'elles soient transmises à l'application mobile lors de sa prochaine connexion.
- Permettre l'interrogation rapide du calendrier toutes les minutes pour déclencher ou arrêter automatiquement la charge.
- Assurer la fiabilité des données, même en cas de redémarrage ou de coupure de courant de l'ESP32.

Structures de données

Voici la structure de la base de données mise en place sur l'ESP32 dans le cadre de ce projet. Elle se compose de deux tables, « SESSIONS » et « ALERTES », stockées dans un fichier « programme.db » sur le système de fichiers LittleFS.

SESSIONS			
	id_session	INTEGER	PRIMARY KEY AUTOINCREMENT
	num_calendrier	INTEGER	
	jours	INTEGER	
	hd	INTEGER	
	md	INTEGER	
	hf	INTEGER	
	mf	INTEGER	

ALERTES			
	id_alerte	INTEGER	PRIMARY KEY
	type_alerte	BOOLEAN	

Table SESSIONS — stocke les sessions de charge programmés par l'utilisateur :

- « id_session » : identifiant unique auto-incrémenté de chaque ligne (inutilisé)
- « num_calendrier » : numéro du calendrier, regroupant une ou plusieurs sessions de charge.
- « jours » : jour de la semaine (1 = lundi, 7 = dimanche)
- « hd » / « md » : heure et minute de début de la session de charge.
- « hf » / « mf » : heure et minute de fin de la session de charge.

Table ALERTES — stocke les alertes générées par le système :

- « id_alerte » : identifiant unique de l'alerte.
- « type_alerte » : type d'alerte (1 = surchauffe, 0 = surintensité).

GitHub

GitHub a été l'outil de gestion de version choisi pour ce projet. Concrètement, chaque membre du groupe travaillait sur sa propre branche correspondant à sa partie du projet (ESP32, application mobile, interface web), puis intégrait ses modifications au code principal une fois validées. Cette organisation nous a permis de travailler en parallèle sans risque d'écraser le travail des autres.

Au cours du développement, cet outil s'est révélé particulièrement utile lors des phases de débogage. En cas de régression, il suffisait de consulter l'historique des commits pour identifier la modification à l'origine du problème et revenir à une version fonctionnelle antérieure. Cette traçabilité a été un atout majeur dans la gestion des crashes rencontrés sur l'ESP32.

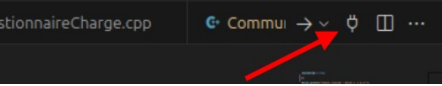


Le dépôt GitHub du projet est accessible à l'adresse suivante :
https://github.com/Corentinh2/PROJET_GESTIONNAIRE_DE_CHARGE

Fiche de test

Cette fiche de test permet de vérifier les principales méthodes de la classe « MemoireProgramme ». La série de tests ainsi menée assurera la validation de leur bon fonctionnement. Afin de faciliter ces vérifications, un menu interactif simulant les interactions de l'application mobile a été créé au sein du moniteur série.

Référence du module testé :		
cas : Piloter la charge		F1.1
Date du test :	30 avril 2026	
Type du test :	Test fonctionnel	
Objectif du test :	Valide le bon fonctionnement des méthodes de la classe « MemoireProgramme »	
<u>Conditions du test</u>		
État initial	Environnement du test	
La base de données Sqlite est présente en local sur l'ESP32, elle contient les tables : SESSIONS, ALERTES.	• ESP32 type lolin32 • poste sous linux mint 22 avec VisualStudio code et platformio. • Moniteur série pour visualiser le bon fonctionnement des méthodes.	
<u>Procédures du test</u>		
N°	Opérations	Résultats attendus

1	- Brancher l'esp32 sur un port usb du pc. - Ouvrir le projet dans visual studio code qui se trouve ici : /home/USERS/ELEVES/CIEL 2024/cfourmond/Documents/CIEL2/Project_2026/Projet_GestionnaireDeCharge/VSCODE/GestionnaireDeCharge_esp32 - Vérifier que la constante « DEBUGETTEST » dans le fichier « constante.h » est à « true » - Ouvrir le monitor série sur le projet via ce bouton :  - Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre> === MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix : </pre>
2	- Saisir le choix numéro 1. - Saisir un calendrier à ajouter dans la base de données avec des heures et minutes de début et de fin proche de l'heure actuelle pour tester le bon fonctionnement de la lecture du calendrier.	Heure qu'il était pour l'exemple : lundi à 10h54 <pre> Votre choix : 1 Saisir trame calendrier (format: jours,hd,md,hf,mf) : jours: 0-127 hd: 0-23 md: 0 ou 30 hf: 0-23 mf: 0 ou 30 Calendrier 1 ajouté avec succès ! Calendrier ajouté : 1,10,55,10,56 </pre> <pre> 27/04/2026 10:55:26 Début de créneau → action début ! </pre> <pre> 27/04/2026 10:56:26 Fin de créneau → action fin ! </pre>

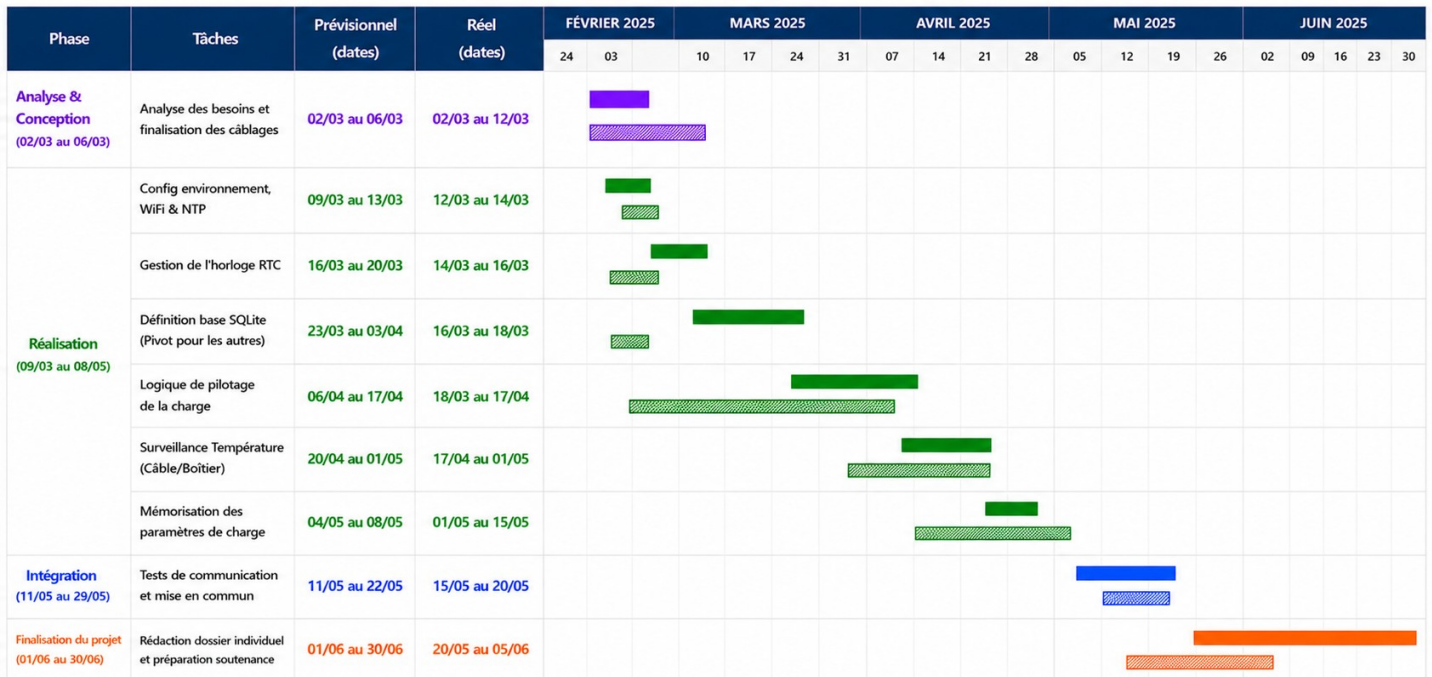
3	Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre> === MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix : </pre>
	Saisir le choix numéro 3.	<pre> Votre choix : 3 Saisir le type d'alerte (1 = alerte température, 0 = alerte courant): </pre>
	Saisir 1 pour simuler un ajout d'une alerte de température dans la base de données et 0 pour une alerte de surintensité pour valider le bon fonctionnement de la méthode « ajouterAlerte ».	Exemple avec une simulation d'alerte de surintensité : <pre> id_alerte = 1 type_alerte = 0 -- </pre>
4	Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre> === MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix : </pre>

	Saisir le choix numéro 5 pour récupérer la trame originale du calendrier rentré dans la base de données précédemment pour valider le bon fonctionnement de la méthode « obtenirTrameOriginal ».	<pre>{"action":"calendrier","id":1,"jours":1,"hd":10,"md":55,"hf":10,"mf":56}</pre>
5	Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre>=== MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix :</pre>
	Saisir le choix numéro 6 pour obtenir les alertes dans la base de données pour valider le bon fonctionnement de la méthode «obtenirAlerte».	<pre>Alertes envoyées : {"action":"alerte","type":"surintensite"}</pre>
6	Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre>=== MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix :</pre>

	Saisir le choix numéro 2.	<pre>id_ligne = 92 id_calendrier = 1 jours = 1 hd = 10 md = 55 hf = 10 mf = 56 ---</pre> Saisir l'id du calendrier à supprimer :
	Saisir l'id du calendrier à supprimer pour valider le bon fonctionnement de la méthode « supprimerCalendrier ».	Calendrier 1 supprimé ! Calendrier(s) restant(s) :
7	Appuyer sur le bouton «BP1» sur la carte pour avoir accès au menu.	<pre>=== MENU === 1 - ajouterCalendrier 2 - supprimerCalendrier 3 - ajouterAlerte 4 - supprimerAlerte 5 - obtenirTrameOriginal 6 - obtenirAlerte 7 - affichercalendrier et alerte Votre choix :</pre>
	Saisir le choix numéro 4.	table alerte vidée

Diagramme de Gantt

Étudiant 1 : Pilotage & Stockage (ESP32) – Planning prévisionnel vs Réel



■ Prévisionnel ■ Réel

Les dates sont incluses (du jour de début au jour de fin).

Le diagramme de Gantt ci-dessus présente le planning prévisionnel et réel de ma partie du projet, découpée en quatre phases principales.

La phase de réalisation s'est globalement bien déroulée dans les délais prévus. On peut noter que la définition de la base de données SQLite a été réalisée bien plus tôt que prévu (18/03 au lieu du 23/03), ce qui a permis aux autres étudiants du groupe de s'appuyer dessus rapidement.

En revanche, la mémorisation des paramètres de charge a pris plus de temps que prévu, s'étendant jusqu'au 15/05 au lieu du 08/05, en raison des ajustements nécessaires sur la structure de la base de données.

Compétences acquises

Le développement de ce gestionnaire de charge connecté a constitué une opportunité idéale pour enrichir et consolider mes compétences techniques et professionnelles, au plus près des exigences actuelles du secteur des systèmes numériques et embarqués.

Dans un premier temps, en matière de technique, le projet m'a permis :

- De manipuler des outils de communication tels que les WebSockets pour établir une communication bidirectionnelle en temps réel entre l'ESP32 et l'application mobile.
- D'utiliser et d'intégrer un format d'échange de données JSON via la bibliothèque ArduinoJson, afin de structurer et de parser les trames échangées entre l'ESP32 et l'application mobile.
- D'implémenter et de gérer une base de données embarquée SQLite sur un système de fichiers LittleFS, en concevant un modèle de données adapté aux contraintes d'un système embarqué à ressources limitées.
- D'interagir avec différents composants matériels tels qu'un capteur de température, une horloge en temps réel ou encore un relais statique.
- De travailler sur un environnement embarqué, en gérant des problématiques propres au temps réel comme la priorité des tâches et la gestion des interruptions.

Dans un second temps, en matière professionnel, ce projet m'a également permis :

- D'appliquer une gestion de projet rigoureuse à travers la planification via un diagramme de Gantt, la répartition des tâches entre les quatre membres de l'équipe et le suivi de l'avancement via GitHub.
- De développer un esprit d'analyse et d'autonomie face aux problèmes techniques rencontrés.
- De pouvoir élaborer une synergie efficace au sein d'une équipe technique.
- De documenter rigoureusement le code, les choix techniques, et les résultats obtenus lors des phases de test.

En définitive, ce projet a constitué une expérience enrichissante et complète, allant de la conception à la réalisation d'un système embarqué communicant. Il m'a permis de mettre en pratique les compétences acquises tout au long de ma formation en BTS CIEL, tout en me confrontant à des problématiques techniques concrètes proches de celles rencontrées dans le milieu professionnel.

Perspectives d'améliorations

Bien que le projet se soit globalement bien déroulé, certains aspects techniques pourraient encore être améliorés. Voici quelques pistes d'optimisation identifiées au cours du développement :

- **Sécurisation de la communication WebSocket** : la communication entre l'application mobile et l'ESP32 ne repose sur aucun mécanisme d'authentification. N'importe quel client connecté au même réseau WiFi pourrait envoyer des commandes à l'ESP32, notamment activer la marche forcée. L'ajout d'un système d'authentification simple par token ou par mot de passe renforcerait la sécurité du système.
- **Validation des données reçues** : les trames JSON reçues depuis l'application mobile sont actuellement insérées en base de données sans vérification préalable des valeurs. Il serait pertinent d'ajouter des contrôles côté ESP32 pour s'assurer que les champs reçus sont cohérents (par exemple vérifier que « hd » et « hf » sont bien compris entre 0 et 23, que le masque de bits « jours » est bien entre 1 et 127, ou encore que l'heure de début est différente de l'heure de fin) avant toute insertion en base de données.
- **Enrichissement de la table ALERTES** : la table « ALERTES » ne stocke actuellement que le type d'alerte. Il serait utile d'y ajouter un horodatage (date et heure de déclenchement) ainsi que la température ou le courant mesuré au moment de l'alerte, afin de permettre à l'utilisateur de consulter un historique complet des incidents thermiques depuis l'application mobile.